

Mock Design: URL Shortener

Phase 4 · Interview-Ready · Estimated time: 1.5 hours

1. Concept From First Principles: Applying the Framework

The URL shortener is the classic warm-up problem — deliberately simple enough to fully solve inside an interview window, yet rich enough to touch encoding schemes, database design, caching, and scale estimation. It's frequently the first problem given specifically because how well you run Chapter 1's framework matters more here than deep specialized knowledge. Treat this chapter as a fully worked transcript of that framework in action.

Step 1 — Clarify requirements

- Functional: given a long URL, generate a short alias; given a short alias, redirect to the original URL. Optionally: custom aliases, expiration dates, click analytics.
- Non-functional: redirects must be low-latency (users feel a slow redirect immediately); the system should prioritize availability over strict consistency (a slightly stale click-count is fine; a failed redirect is not).

Step 2 — Estimate scale

Assume 100 million new URLs shortened per month (~40 writes/second average) and a 100:1 read:write ratio typical of link-sharing products, giving ~4,000 redirects/second average. At 7 characters using base62 encoding (a–z, A–Z, 0–9 = 62 symbols), $62^7 \approx 3.5$ trillion possible short codes — comfortably enough for years of growth. Storage: assuming ~500 bytes per URL record (original URL, short code, metadata), 100M/month $\approx$ 50GB/month, trivial for modern storage but useful to state out loud.

Step 3 — API

POST /shorten {longUrl, customAlias?, expiresAt?} -> {shortUrl} and GET /{shortCode} -> 302 redirect to longUrl. Using a **302 (temporary) redirect** rather than 301 (permanent) is a deliberate, discussable choice: 301 lets browsers cache the redirect client-side, reducing server load, but makes click analytics and future alias changes impossible since the browser stops calling your server at all — a real trade-off worth naming unprompted.

Step 4 — High-level design

Client → Load Balancer (Phase 1, Ch 6) → API service → on write: generate a unique short code and write to the database; on read: look up the short code, resolve it to the long URL, return a redirect. Given the 100:1 read-heavy ratio, put a cache (Phase 1, Ch 8) — Redis, with an LRU eviction policy — in front of the database for the hot subset of short codes, since most traffic concentrates on recently-created or viral links (Zipfian distribution, the same pattern behind Phase 1's caching chapter). The database itself is a simple key-value-shaped table (short_code, long_url, created_at, expires_at) — well suited to a NoSQL key-value store (Phase 2, Ch 1) given the lookup pattern is always by exact short_code, with no complex relational queries needed.

Step 5 — Deep dive: short code generation

This is where nearly every interviewer steers, because it has multiple valid approaches with real trade-offs. **Approach A — hash + truncate:** MD5/SHA-256 hash the long URL, take the first 7 base62 characters. Simple, but collisions are possible and must be detected and retried, and identical URLs always produce identical short codes (which may or may not be desired). **Approach B — auto-incrementing counter + base62 encode:** a distributed counter (backed by, e.g., a dedicated counter service or pre-allocated ID ranges per server, directly reusing Phase 2's distributed ID generation concepts) generates a unique integer, then base62-encodes it into a short code. This guarantees no collisions by construction, but requires a coordinated, reliable counter service that itself must scale and stay highly available — a real operational cost worth naming. Most production systems (and most strong interview answers) favor Approach B specifically because collision-handling logic under high write concurrency is a genuinely awkward failure mode to reason about correctly under interview time pressure.

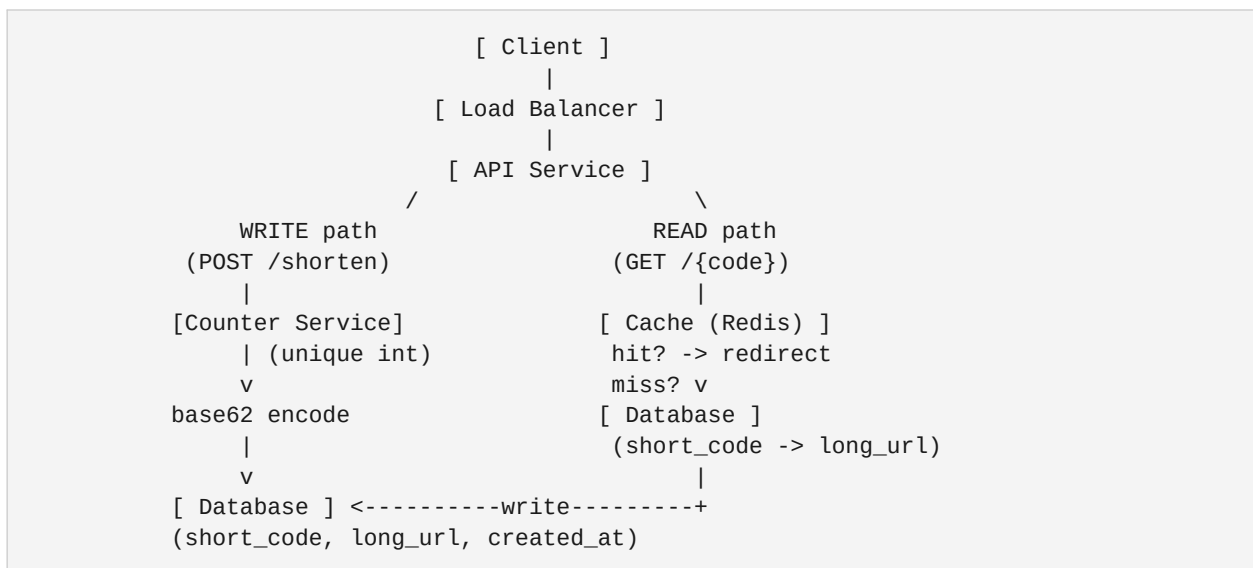
Step 6 — Trade-offs and wrap-up

Volunteer: the cache is a single point of read-path acceleration but not correctness — a cache miss always falls through correctly to the database, so cache failure degrades latency, not availability. Name monitoring: redirect latency p99, cache hit rate, and counter-service availability as the three metrics you'd dashboard first (Phase 3, Ch 13).

2. Real-World Example / Case Study

Bitly and TinyURL are the canonical real-world examples of this exact system, and this problem is a frequent early-round warm-up at Indian Tier-1 product companies (Flipkart, Swiggy) specifically because it reliably separates candidates who run a structured process from those who don't, without requiring deep specialized domain knowledge — making it a fair, low-noise signal for a first system design round.

3. Diagram



4. Trade-offs Table

Decision	Alternative	Why this choice
Counter + base62 encode	Hash + truncate (MD5/SHA)	Guarantees no collisions by construction, avoids retry logic
302 redirect	301 redirect	Preserves click analytics and future alias edits; 301 blocks both via browser caching
Redis cache in front of DB	No cache, DB handles all reads	Read:write is 100:1; cache absorbs the hot Zipfian subset
Key-value store for storage	Relational database	Access pattern is always exact-match lookup by short_code

5. Common Interview Questions

Q1: "How do you guarantee short codes are unique across multiple API servers writing concurrently?"

Model approach: Explain the counter-service approach with pre-allocated ID ranges per server (each server claims a batch of, say, 1,000 IDs at a time), avoiding a single shared counter becoming a bottleneck while still guaranteeing global uniqueness — directly reusing Phase 2's distributed ID generation pattern.

Q2: "What happens if the cache goes down entirely?"

Model approach: State clearly that redirects still work correctly by falling through to the database — the cache affects latency, not correctness — but note the database must now absorb 100% of read traffic, so you'd want auto-scaling or read replicas (Phase 1, Ch 3/8) as a safety net for that scenario.

Q3: "How would you support custom aliases without breaking uniqueness guarantees?"

Model approach: Custom aliases bypass the counter-generated path entirely; check the requested alias against the database with a unique constraint on short_code, and return a clear conflict error if taken — a much simpler uniqueness problem than the auto-generated case since it's a single synchronous check, not a coordinated generation scheme.

6. Practice Exercises

- 1 Run this entire six-step framework yourself, out loud, with a timer, for "design a URL shortener," without looking at this chapter.
- 2 Design the schema and sketch the base62 encoding function for turning an auto-incrementing integer ID into a 7-character short code.
- 3 Extend the design to support link expiration: what changes in the schema, the read path, and the write path?

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design Mock Interview: Design URL Shortener"	Full live mock interview walkthrough matching this chapter's structure.
English	"Design a URL Shortener (Bitly) - System Design Interview"	Clear coverage of the counter-vs-hash short-code generation trade-off.
Hindi	"TinyURL System Design URL Shortener System Design Interview Question Bitly System Design"	Hindi/Hinglish walkthrough of this exact classic problem.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering this and other warm-up-tier problems.

8. Key Takeaways

- The URL shortener tests process discipline (Chapter 1's framework) more than deep specialized knowledge — it's a common, deliberate warm-up problem.
- Counter + base62 encoding is generally preferred over hash + truncate because it guarantees uniqueness without retry logic.
- 302 (temporary) redirects preserve analytics and alias flexibility; 301 (permanent) redirects trade that away for client-side caching.
- A read-heavy (100:1) access pattern justifies a cache whose failure degrades latency, not correctness.
- Key-value storage fits this problem's exact-match lookup pattern better than a relational database.
- Custom aliases are a simpler uniqueness problem than auto-generated codes — a single synchronous constraint check, not coordinated generation.
- This chapter is the first fully worked application of Chapter 1's six-step framework — every remaining mock-design chapter follows the same shape.